最大流詳解

26-E01 反平行流、淨流與殘餘

推導

a入3+1、出3+1；b入1+3、出1+3，皆4，source淨出4、sink淨入4。淨f_ab=3−1=2、f_ba=−2；其他方向為原g及其負反向。因此c_f(a,b)=3−2=1、c_f(b,a)=2−(−2)=4。

Residual路s→a→b→t三邊剩餘皆1，增廣1後f_sa=4、f_sb=1、f_ab=3、f_at=1、f_bt=4。對應非負arc流可取g_ab=3、g_ba=0，其餘同正淨流。a收4送3+1；b收1+3送4，值5。

結論與檢核

Cut S={s}容量4+1=5，等於可行流，故最優5。反向殘餘4不僅是「舊g_ab的3」，也不是負流錯誤；它由原反向容量與淨流共同決定。

26-E02 流與割是不同方向的證書

推導

取f_sa=4、f_sb=3、f_ab=1、f_at=3、f_bt=4。a收4送1+3，b收3+1送4，容量全合法，值7。取S={s,a,b}、T={t}，容量a→t的3加b→t的4為7，故最優7。

結論與檢核

只有端點總和給≤min(8,7)=7，沒有給≥7。可把中間網路改成所有路都要過容量1的瓶頸，保持同樣端點總和卻僅能送1。因此需可行流的下界證書，不能由端點上界直接判定等號。

26-E03 偽多項式的實際選路族

推導

第k對完成後，(f_sa,f_sb,f_ab,f_at,f_bt)=(k,k,0,k,k)，k=0,…,C。若k<C，第一路的a→b殘餘1，其他邊至少1，故δ=1，變(k+1,k,1,k,k+1)。第二路使用b→a殘餘1撤回，變(k+1,k+1,0,k+1,k+1)。每對增值2，直到第C對值2C，source邊全飽和，故恰2C次。

C=2^B時，常數條邊容量各用B+1位，輸入長Θ(B)，輪數2^(B+1)對輸入長度指數大。這由實際軌跡證明，不是僅由一個O(C)上界猜下界。

結論與檢核

BFS先選s→a→t或s→b→t，各瓶頸C，兩次便到2C。多出a→b的跨邊不在任何最短兩邊路徑上；容量數值大不會增加此例的BFS增廣次數。

最大流詳解 （續）

26-E04 Matching、翻轉與 Hall 障礙

推導

由未配L2出發，交替路L2→R1→L1→R2→L3→R3，反向段撤回L1R1、L3R2，加入L2R1、L1R2、L3R3，大小3。右側只有3點，任何matching≤3，故最大3。取全部左點{L1,L2,L3}為cover，大小3；matching下界也為3，所以是最小cover。

刪L3R3後，A={L1,L2,L3}的鄰居只剩{R1,R2}，|N(A)|=2<3。三個左點無法配到三個不同右點。大小2仍可行，例如L2R1、L3R2，故最大2。

結論與檢核

交替路翻轉控制的是所有中間點的端點互斥，不是單獨替換一條邊。Hall失敗集合給出無法全配的結構性證書。

26-E05 沒有殘餘 s-t 路仍可能未算完

推導

初始化f_su=2、f_ut=0，e(s,u,t)=(−2,2,0)，h=(3,0,0)。殘餘u→s=2、u→t=1，source無正殘餘出邊，故沒有s-t殘餘路。但u不守恆，這只是preflow，不符合max-flow min-cut對「可行flow」的前提。

u無admissible出邊，relabel到1；沿u→t推1，e變(−2,1,1)，u→t飽和。剩餘可用鄰居為s的height3，所以u重標4；沿u→s退1，e變(−1,0,1)，f_su=1、f_ut=1，完成。

結論與檢核

最後為可行流1，cut S={s,u}容量1證最優。初始source推出2不等於可行流值已達2；先恢復內點守恆才能使用流／割等價。

26-E06 改變鄰居順序，不改最優值

推導

初始e(a,b,c,t)=(4,3,2,0)。a重標1，先a→t推1得(3,3,2,1)，再a→b推2得(1,5,2,1)；重標6、退s一單位得(0,5,2,1)。a結束L仍[a,b,c]、height(6,0,0)、current(s,t,t)。

b重標1，先b→t推4得(0,1,2,5)，再b→c推1得(0,0,3,5)。結束move-front，L=[b,a,c]、height(6,1,0)、current(s,c,t)。再掃a無動作，c重標1後送t三單位得(0,0,0,8)，L=[c,b,a]、height(6,1,1)、current(s,c,t)。再掃b、a後終止。

最後原邊流sa=3、sb=3、sc=2、ab=2、at=1、bc=1、bt=4、ct=3。a收3送3，b收5送5，c收3送3。Cut S={s,a}容量3+2+2+1=8，證最優。

最大流詳解 （續）

結論與檢核

正文為bc=2、bt=3、ct=4，此題為1,4,3；flow分配可不同，最優值同為8。Current仍停在清空excess的那次push所用鄰居，不自動移到下一項。

26-E07 Current 的安全性與成本

推導

uv殘餘從0恢復，只能因vu反向push；此時h(v)=h(u)+1，恢復的uv向上走，不能admissible。若先前殘餘正而不admissible，有效整數標號排除h(u)>h(v)+1，又非相等，故h(u)≤h(v)。u不變、v只升高，仍無法有h(u)=h(v)+1。

所以只在u自身relabel後重設head即可。u重標O(n)次，每段最多掃deg(u)個鄰居，總O(n deg(u))；所有u相加O(nΣdeg)=O(nm)。

結論與檢核

若每次discharge都重設，即使height未改也會重掃先前略過的前綴，便不能再用「每個height階段只掃一次」支付；這不立即證算法錯誤，但原O(nm)掃描成本論證已不適用。

26-E08 有理容量與精確證書

推導

沿s→a→t送1/3、s→b→t送1/6，總1/2。Cut S={s,a}容量s→b的1/6加a→t的1/3=1/2，故最優。乘6後容量sa=3、at=2、sb=bt=1，最大流3，還原3/6=1/2。

一般分母乘積D的bit長≤分母bit長總和，為輸入長度O(N)；但D本身可為2^Θ(N)。整數化後Ford–Fulkerson的F*也可能數值很大，有限終止不等於多項式輪數。Edmonds–Karp以每邊O(n)次critical界限制總O(nm)輪，與D數值無關。

結論與檢核

用共分母整數運算可精確核對流／割，不必靠浮點差很小就宣稱相等。有限輪數與每輪位元成本都受多項式控制，才完成有理輸入的多項式時間論證。

26-E09 把路徑補成完整殘餘網路

單位容量的第1輪原邊流量依(sa,sb,ab,at,bt)為(1,0,1,0,1)。正殘餘弧恰為a→s:1、s→b:1、b→a:1、a→t:1、t→b:1。第2輪走s→b→a→t，將ab原流量退回0，得到(1,1,0,1,1)。正殘餘弧恰為a→s:1、b→s:1、a→b:1、t→a:1、t→b:1。

最終a、b各收1送1，可行流值2。割{s}原容量1+1=2，故最優。雖然a→b殘餘為1，但s沒有正殘餘出弧，無法到a，不能再形成source–sink增廣路。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

最大流詳解 （續）

容量(3,2,1,2,3)時，完整狀態如下；每列是本輪增廣前，數值全為殘餘容量。

輪次	全部正殘餘弧	d(a,b,t)、路徑、δ
1	sa3、sb2、ab1、at2、bt3	(1,1,2)，s-a-t，2
2	sa1、as2、sb2、ab1、ta2、bt3	(1,1,2)，s-b-t，2
3	sa1、as2、bs2、ab1、ta2、bt1、tb2	(1,2,3)，s-a-b-t，1
終止	as3、bs2、ba1、ta2、tb3	(∞,∞,∞)，無路

流值依次0、2、4、5；最後原流(sa,sb,ab,at,bt)=(3,2,1,2,3)。a收3送1+2；b收2+1送3。割{s}容量3+2=5，完成獨立最優性檢核。

26-E10 格點不能共用的逃生模型

只限制邊時，兩條路分別為(3,2)→(3,3)→(3,4)→(3,5)與(2,3)→(3,3)→(4,3)→(5,3)。邊各用一次，所以可送2；source連兩起點的總容量2給相同上界。兩條路卻共用z=(3,3)，因此不符合格點不共用的規格。

拆每個開放格v成v_in、v_out，內部邊容量1；相鄰格以兩方向的out→in容量1連接。source至每個起點_in、每個開放邊界_out至sink的容量皆1。此例取S包含s、u與v的兩側節點及z_in，其餘在T。唯一S→T弧為z_in→z_out，容量1，所以任何流至多1。第一條路可送1，故恰為1。

一般有q個可走格、p個不同起點、b個開放邊界，頂點N=2q+2。內部邊q條，移動有向邊至多4k(k−1)條，再加p+b，故M≤q+4k(k−1)+p+b。障礙格不加入；起點必為可走格。整數最大流可去除循環後分解成單位source–sink路徑；內部容量1禁止兩路共用格點，值p又迫使每個起點都送出1。反向由每條格點不相交路送1即得合法流，所以兩種可行解等價。

k=1且唯一格開放時，邊界數為1，不能代4k−4=0。有一人時走s→v_in→v_out→t，最大流1；無人則0。一般起點已在邊界也可直接經該格內部邊離開，不必先移到鄰格。封閉格不能作起點；若規格另允許共用起點或出口，內部容量與對應證明都須隨限制調整。

最大流詳解 （續）

26-E11 六點網路的反向撤流與割證書

以下流量向量順序固定為(sw,wx,xy,xt,sy,yz,zt)：

輪次	路徑上的正殘餘容量	δ	更新後流量向量	流值
1	(8,9,3,7,8)	3	(3,3,3,0,0,3,3)	3
2	(10,3,4)	3	(3,3,0,3,3,3,3)	6
3	(7,4,5)	4	(3,3,0,3,7,7,7)	10
4	(5,6,1)	1	(4,4,0,4,7,7,7)	11

第1輪送過xy後，殘餘y→x由0變3。第2輪將它用完，原f(x,y)回到0；該輪增加sy與xt各3，維持x、y守恆，原有sw、wx、yz、zt流量不變。初始c(y,x)=0且f(y,x)=0，所以r(y,x)=0；一開始選s-y-x-t會使用不存在於正殘餘網路的弧。

最後正殘餘弧為sw4、ws4、wx5、xw4、xy3、sy3、ys7、tx4、zy7、zt1、tz7。由s沿sw、wx、sy、xy只能到{s,w,x,y}；xy是原方向的剩餘容量，不是之前的撤流殘餘yx。跨割原邊只有xt:4與yz:7，容量11。內點w、x各收送4，y、z各收送7，流值等於source淨出4+7及sink淨入4+7，也等於割11，因此最優。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最大流詳解 （續）

原創驗算程式附錄: flow_revision.py

```
"""Sparse exact net-flow models. Traces count operations, not measured speed."""
from collections import deque

class Network:
    def __init__(self,vertices,capacities,source,sink,neighbor_order=None):
        self.vertices=list(vertices);self.s=source;self.t=sink
        self.c={edge:value for edge,value in capacities.items() if value>0};self.f={}
        self.neighbors={u:[] for u in self.vertices}
        for u,v in self.c:
            if v not in self.neighbors[u]:self.neighbors[u].append(v)
            if u not in self.neighbors[v]:self.neighbors[v].append(u)
        if neighbor_order:
            for u,order in neighbor_order.items():
                if set(order)!=set(self.neighbors[u]):raise ValueError('neighbor order must
contain exactly original/reverse neighbors')
                self.neighbors[u]=list(order)
        self.e={u:0 for u in self.vertices};self.h={u:0 for u in self.vertices};self.current=
{u:0 for u in self.vertices}

    def residual(self,u,v):return self.c.get((u,v),0)-self.f.get((u,v),0)
    def send(self,u,v,amount):
        assert 0<amount<=self.residual(u,v)
        self.f[u,v]=self.f.get((u,v),0)+amount;self.f[v,u]=self.f.get((v,u),0)-amount
        self.e[u]-=amount;self.e[v]+=amount
    def value(self):return -self.e[self.s]
    def check(self,preflow=False,height=False):
        for u in self.vertices:
            assert self.e[u]==sum(self.f.get((v,u),0) for v in self.neighbors[u])
            if u!=self.s:assert self.e[u]>=0 if preflow else (u==self.t or self.e[u]==0)
            for v in self.neighbors[u]:
                assert self.f.get((u,v),0)==-self.f.get((v,u),0)
                assert self.f.get((u,v),0)<=self.c.get((u,v),0)
                if height and self.residual(u,v)>0:assert self.h[u]<=self.h[v]+1
        assert sum(self.e.values())==0
        if height:assert self.h[self.s]==len(self.vertices) and self.h[self.t]==0
    def reachable(self):
        seen={self.s};q=deque([self.s])
        while q:
            u=q.popleft()
            for v in self.neighbors[u]:
                if v not in seen and self.residual(u,v)>0:seen.add(v);q.append(v)
        return seen
    def cut_capacity(self,side):return sum(c for (u,v),c in self.c.items() if u in side and v
not in side)

def edmonds_karp(net):
    trace=[]
    while True:
```

最大流詳解 （續）

```
        parent={net.s:None};distance={net.s:0};q=deque([net.s])
        while q:
            u=q.popleft()
            for v in net.neighbors[u]:
                if v not in parent and
net.residual(u,v)>0:parent[v]=u;distance[v]=distance[u]+1;q.append(v)
        if net.t not in parent:return net.value(),trace
        path=[];v=net.t
        while v!=net.s:u=parent[v];path.append((u,v));v=u
        path.reverse();d=min(net.residual(u,v) for u,v in path)
        for u,v in path:net.send(u,v,d)

net.check();trace.append({'path':path,'delta':d,'distance':distance,'flow':dict(net.f)})

def relabel_to_front(net,trace_enabled=False):
    n=len(net.vertices);net.h[net.s]=n
    for v in net.neighbors[net.s]:
        amount=net.c.get((net.s,v),0)
        if amount:net.send(net.s,v,amount)
    linked=[u for u in net.vertices if u not in (net.s,net.t)];index=0;trace=[]
    def snapshot(action,u=None,v=None,amount=None):
        if
trace_enabled:trace.append({'action':action,'u':u,'v':v,'amount':amount,'e':dict(net.e),'h':d
ict(net.h),'current':{x:(net.neighbors[x][net.current[x]] if net.current[x]
<len(net.neighbors[x]) else None) for x in linked},'list':list(linked)})
    net.check(True,True);snapshot('initialize')
    while index<len(linked):
        u=linked[index];old=net.h[u]
        while net.e[u]>0:
            if net.current[u]==len(net.neighbors[u]):
                net.h[u]=1+min(net.h[v] for v in net.neighbors[u] if net.residual(u,v)>0)
                net.current[u]=0;net.check(True,True);snapshot('relabel',u)
            else:
                v=net.neighbors[u][net.current[u]]
                if net.residual(u,v)>0 and net.h[u]==net.h[v]+1:

amount=min(net.e[u],net.residual(u,v));net.send(u,v,amount);net.check(True,True);snapshot('pu
sh',u,v,amount)
                else:net.current[u]+=1
        if net.h[u]>old:linked.remove(u);linked.insert(0,u);snapshot('move-front',u)
        index=linked.index(u)+1
    net.check(False,True);return net.value(),trace

CAPACITY={('s','a'):4,('s','b'):3,('s','c'):2,('a','b'):2,('a','t'):1,('b','c'):2,
('b','t'):4,('c','t'):5}
ORDER={'a':['b','t','s'],'b':['c','t','a','s'],'c':['t','b','s']}
if __name__=='__main__':
    network=Network(['s','a','b','c','t'],CAPACITY,'s','t',ORDER)
    value,trace=relabel_to_front(network,True)
    for row in trace:print(row)
    side=network.reachable();print('value',value,'cut',side,network.cut_capacity(side))
```

最大流詳解 （續）

原創驗算程式附錄: relabel_to_front_trace.py

```
"""Run the standard all-vertices relabel-to-front schedule on a small network."""

from dataclasses import dataclass, field


VERTICES = ("s", "a", "b", "c", "t")
CAPACITY = {
    ("s", "a"): 4,
    ("s", "b"): 3,
    ("s", "c"): 2,
    ("a", "b"): 2,
    ("a", "t"): 1,
    ("b", "c"): 2,
    ("b", "t"): 4,
    ("c", "t"): 5,
}
NEIGHBORS = {
    "a": ("b", "t", "s"),
    "b": ("c", "t", "a", "s"),
    "c": ("t", "b", "s"),
}


@dataclass
class Vertex:
    name: str
    height: int = 0
    excess: int = 0
    current: int = 0
    flow: dict = field(default_factory=dict)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最大流詳解 （續）

```
@dataclass
class Network:
    source: Vertex
    sink: Vertex
    vertices: dict

    def net_flow(self, u, v):
        return self.vertices[u].flow.get(v, 0)

    def residual(self, u, v):
        return CAPACITY.get((u, v), 0) - self.net_flow(u, v)

    def push(self, u, v):
        amount = min(self.vertices[u].excess, self.residual(u, v))
        assert amount > 0
        self.vertices[u].flow[v] = self.net_flow(u, v) + amount
        self.vertices[v].flow[u] = self.net_flow(v, u) - amount
        self.vertices[u].excess -= amount
        self.vertices[v].excess += amount


def state(net):
    return tuple(net.vertices[name].excess for name in ("a", "b", "c", "t"))


def heights(net):
    return tuple(net.vertices[name].height for name in ("a", "b", "c"))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最大流詳解 （續）

```
def discharge(net, name):
    u = net.vertices[name]
    old_height = u.height
    rows = []
    while u.excess > 0:
        neighbors = NEIGHBORS[name]
        if u.current == len(neighbors):
            u.height = 1 + min(
                net.vertices[v].height
                for v in neighbors
                if net.residual(name, v) > 0
            )
            u.current = 0
            rows.append(("relabel", state(net), heights(net)))
            continue
        v = neighbors[u.current]
        if net.residual(name, v) > 0 and u.height == net.vertices[v].height + 1:
            net.push(name, v)
            rows.append((f"push->{v}", state(net), heights(net)))
        else:
            u.current += 1
    return u.height > old_height, rows


def run():
    vertices = {name: Vertex(name) for name in VERTICES}
    vertices["s"].height = len(VERTICES)
    net = Network(vertices["s"], vertices["t"], vertices)
    for target, capacity in (("a", 4), ("b", 3), ("c", 2)):
        vertices["s"].flow[target] = capacity
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

最大流詳解 （續）

```
        vertices[target].flow["s"] = -capacity
        vertices[target].excess += capacity
        vertices["s"].excess -= capacity

    assert state(net) == (4, 3, 2, 0)
    assert heights(net) == (0, 0, 0)
    linked = ["a", "b", "c"]
    index = 0
    while index < len(linked):
        name = linked[index]
        increased, _ = discharge(net, name)
        if increased:
            linked.remove(name)
            linked.insert(0, name)
        index = linked.index(name) + 1

    assert linked == ["c", "b", "a"]
    assert state(net) == (0, 0, 0, 8)
    assert heights(net) == (6, 1, 1)
    assert sum(net.net_flow("s", v) for v in ("a", "b", "c")) == 8
    assert vertices["s"].excess == -8
    assert sum(v.excess for v in vertices.values()) == 0
    print("PASS relabel-to-front list=c,b,a excess=(0,0,0,8) heights=(6,1,1)")
    return linked, state(net)


if __name__ == "__main__":
    run()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study